

Kubernetes Fundamentals

ReplicaSet & Deployment

A complete, practical introduction to the two most important Kubernetes workload controllers. Deploy, break, heal, scale, update, and roll back real workloads — step by step, with clear commands and expected output at every stage.

Level	Beginner → Intermediate
Time	50 – 70 minutes
Needs	Any Kubernetes cluster (Minikube, kind, or managed)
Covers	ReplicaSet auto-healing · Deployment rollouts · rollback · scaling

What's Inside

1. Kubernetes Workloads at a Glance
2. Before You Start
3. PART A — ReplicaSet
 - A1 · Create a Namespace
 - A2 · Write the ReplicaSet Manifest
 - A3 · Deploy & Verify
 - A4 · Auto-Healing in Action
 - A5 · Scaling the ReplicaSet
4. PART B — Deployment
 - B1 · Write the Deployment Manifest
 - B2 · Deploy & Verify
 - B3 · Rolling Update
 - B4 · Rollback
 - B5 · Scaling the Deployment
5. ReplicaSet vs. Deployment — Side by Side
6. Validation Checklist
7. Troubleshooting Guide
8. Cleanup
9. Recap & Quiz

1 · Kubernetes Workloads at a Glance

Kubernetes runs your applications inside **Pods**. But Pods on their own don't recover if something goes wrong — that job belongs to **controllers**. This lab covers the two controllers you will use most:

Object	Purpose
ReplicaSet	Keeps a fixed number of identical Pods running at all times (auto-healing).
Deployment	Manages ReplicaSets for you, adding rolling updates, versioning, and rollback.

In practice, you will almost always use a Deployment, not a bare ReplicaSet — but understanding ReplicaSet first makes it much easier to understand what a Deployment is really doing underneath.

■ HOW THIS LAB IS STRUCTURED

Part A builds a ReplicaSet directly, so you can see auto-healing in its purest form. **Part B** builds an equivalent Deployment, so you can see the extra capabilities it adds on top.

2 · Before You Start

You just need two things:

What	Why
kubectl	The command-line tool used to talk to your cluster
A running cluster	Minikube, kind, or any cluster you already have access to

Quick check — confirm kubectl can reach your cluster:

```
$ kubectl get nodes
NAME STATUS ROLES AGE
minikube Ready control-plane 2d
```

■ **TIP**

If your node shows Ready, you're all set to begin.

PART A · REPLICASET

A **ReplicaSet** is a simple Kubernetes rule: "always keep this many copies of my Pod running." If a Pod disappears, Kubernetes notices and creates a new one — automatically. That is **auto-healing**.

In Part A you will:

- Create 3 copies of a simple nginx web server Pod.
- Delete one Pod on purpose, as if it had crashed.
- Watch Kubernetes create a new one automatically.
- Change the replica count and watch it converge again.

A 1

Create a Namespace

A namespace keeps this lab's resources separate from everything else, so cleanup is a single command later.

```
$ kubectl create namespace k8s-lab
namespace/k8s-lab created
```

```
$ kubectl config set-context --current --namespace=k8s-lab
Context "minikube" modified.
```



Write the ReplicaSet Manifest

Create a file named `web-rs.yaml`. This declares 3 running copies of an nginx web server:

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web-frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-frontend
  template:
    metadata:
      labels:
        app: web-frontend
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
          ports:
            - containerPort: 80
```

■ WHAT MATTERS HERE

replicas: 3 is the number Kubernetes will always try to keep running. **selector** defines which Pods belong to this ReplicaSet. **template** is the blueprint used whenever a new Pod needs to be created.

A
3

Deploy & Verify

```
$ kubectl apply -f web-rs.yaml
replicaset.apps/web-frontend created
```

Check that 3 Pods came up:

```
$ kubectl get pods
NAME READY STATUS AGE
web-frontend-a1b2c 1/1 Running 10s
web-frontend-c3d4e 1/1 Running 10s
web-frontend-e5f6g 1/1 Running 10s
```

Check the ReplicaSet summary:

```
$ kubectl get replicaset
NAME DESIRED CURRENT READY AGE
web-frontend 3 3 3 12s
```

■ GOOD SIGN

DESIRED, CURRENT, and READY all show 3 — the fleet is healthy.

A
4

Auto-Healing in Action

Pick any Pod name from above and delete it, simulating a crash:

```
$ kubectl delete pod web-frontend-a1b2c
pod "web-frontend-a1b2c" deleted
```

Check your Pods again immediately:

```
$ kubectl get pods
NAME READY STATUS AGE
web-frontend-c3d4e 1/1 Running 1m
web-frontend-e5f6g 1/1 Running 1m
web-frontend-h7i8j 1/1 Running 2s
```

■ THIS IS AUTO-HEALING

A brand-new Pod appeared within seconds. You never created it — the ReplicaSet controller noticed the count dropped to 2 and fixed it back to 3 automatically.

A
5

Scaling the ReplicaSet

Auto-healing always aims at whatever number you set. Change that number and watch it converge:

```
$ kubectl scale replicaset web-frontend --replicas=5
replicaset.apps/web-frontend scaled
```

```
$ kubectl get replicaset
NAME DESIRED CURRENT READY AGE
web-frontend 5 5 5 3m
```

Put it back to 3 before moving on:

```
$ kubectl scale replicaset web-frontend --replicas=3
replicaset.apps/web-frontend scaled
```

■ A REPLICASET'S LIMIT

Notice that nothing here changed the *nginx version* — a ReplicaSet has no idea how to update an image safely. That gap is exactly what a Deployment fills, next.

PART B · DEPLOYMENT

A **Deployment** manages ReplicaSets for you. It has the exact same auto-healing behavior as Part A, plus the ability to safely change your application's image version — a **rolling update** — and undo that change if something breaks — a **rollback**.

In Part B you will:

- Create a Deployment running the same web server, at version 1.25.
- Update it live to version 1.27 and watch a rolling update happen.
- Roll back to the previous version with one command.
- Scale the Deployment up and down.

First, remove the bare ReplicaSet from Part A so it doesn't compete for the same Pods:

```
$ kubectl delete replicaset web-frontend
replicaset.apps "web-frontend" deleted
```



Write the Deployment Manifest

Create a file named `web-deployment.yaml`. Notice how similar it is to the ReplicaSet from Part A:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-frontend
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: web-frontend
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          ports:
            - containerPort: 80
```

■ WHAT'S NEW HERE

Everything above **strategy** is identical to a ReplicaSet. **strategy.type: RollingUpdate** tells the Deployment to replace Pods gradually. **maxSurge / maxUnavailable** control how fast and how safely that happens.

B
2

Deploy & Verify

```
$ kubectl apply -f web-deployment.yaml
deployment.apps/web-frontend created
```

Check the Deployment, its ReplicaSet, and its Pods:

```
$ kubectl get deployment
NAME READY UP-TO-DATE AVAILABLE AGE
web-frontend 3/3 3 3 20s
```

```
$ kubectl get replicaset
NAME DESIRED CURRENT READY AGE
web-frontend-6b7d9f8c6d 3 3 3 20s
```

■ KEY OBSERVATION

You created a Deployment, but Kubernetes created a **ReplicaSet** for you automatically (with a hash suffix), and that ReplicaSet created the Pods. Three layers, one command.

B
3

Rolling Update

Update the image to a newer version — this is a real deployment upgrade:

```
$ kubectl set image deployment/web-frontend nginx=nginx:1.27-alpine
deployment.apps/web-frontend image updated
```

Watch the rollout happen live:

```
$ kubectl rollout status deployment/web-frontend
Waiting for deployment "web-frontend" rollout to finish: 1 out of 3 new replicas...
Waiting for deployment "web-frontend" rollout to finish: 2 out of 3 new replicas...
deployment "web-frontend" successfully rolled out
```

A second ReplicaSet now exists for the new version:

```
$ kubectl get replicaset
NAME DESIRED CURRENT READY AGE
web-frontend-6b7d9f8c6d 0 0 0 4m # old version
web-frontend-8f4c6a9b2e 3 3 3 15s # new version
```

■ THIS IS THE MAGIC OF A DEPLOYMENT

It created a brand-new ReplicaSet for version 1.27, scaled it up gradually, and scaled the old ReplicaSet down to zero — all without downtime, because **maxUnavailable: 0** guaranteed at least 3 Pods were always available.



Rollback

Imagine version 1.27 turned out to be broken. Undo it with a single command:

```
$ kubectl rollout undo deployment/web-frontend
deployment.apps/web-frontend rolled back
```

Confirm you're back on the old image:

```
$ kubectl get deployment web-frontend -o
jsonpath="{.spec.template.spec.containers[0].image}"
nginx:1.25-alpine
```

You can also see the full rollout history:

```
$ kubectl rollout history deployment/web-frontend
REVISION  CHANGE-CAUSE
1  <none>
2  <none>
3  <none>
```

■ WHY THIS MATTERS

This is the single biggest reason production teams use Deployments instead of bare ReplicaSets — a bad release is one command away from being undone.



Scaling the Deployment

Scaling works exactly the same way as a ReplicaSet:

```
$ kubectl scale deployment web-frontend --replicas=5
deployment.apps/web-frontend scaled
```

```
$ kubectl get deployment
NAME READY UP-TO-DATE AVAILABLE AGE
web-frontend 5/5 5 5 10m
```

Scale back down to 3:

```
$ kubectl scale deployment web-frontend --replicas=3
deployment.apps/web-frontend scaled
```

5 · ReplicaSet vs. Deployment — Side by Side

Capability	ReplicaSet	Deployment
Keeps N Pods running (auto-heal)	Yes	Yes (via its ReplicaSet)
Manual scaling	Yes	Yes
Safe image updates (rolling update)	No	Yes
Rollback to a previous version	No	Yes
Keeps update history	No	Yes
Recommended for everyday use	Rarely used directly	Yes — the standard choice

In short: a Deployment *is* a ReplicaSet, plus the missing piece — a safe way to change what's running without downtime.

6 · Validation Checklist

ReplicaSet reached DESIRED = CURRENT = READY = 3	■
Deleting a Pod triggered automatic replacement	■
Scaling the ReplicaSet up and back down worked as expected	■
Deployment created its own ReplicaSet automatically	■
kubectl set image triggered a rolling update with zero downtime	■
A new ReplicaSet appeared for the updated version, old one scaled to 0	■
kubectl rollout undo restored the previous image version	■
Scaling the Deployment up and back down worked as expected	■

7 · Troubleshooting Guide

Symptom	Likely Cause	Check This
Pods stuck Pending	Not enough CPU/memory on the node	<code>kubectl describe pod</code>
READY never reaches 1/1	Container failing its health check	<code>kubectl logs</code>
Rollout stuck mid-way	New image can't start (bad tag, crash loop)	<code>kubectl rollout status</code> ; <code>kubectl describe pod</code>
Old Pods never terminate	<code>maxUnavailable: 0</code> needs extra capacity to surge	<code>kubectl get rs</code> — check if new RS can schedule
Scale command has no effect	Wrong object name or namespace	<code>kubectl get deployment -A</code>

8 · Cleanup

Remove everything this lab created with one command:

```
$ kubectl delete namespace k8s-lab
namespace "k8s-lab" deleted
```

Switch back to your normal namespace:

```
$ kubectl config set-context --current --namespace=default
Context "minikube" modified.
```

9 · Recap & Quiz

What you learned:

- A ReplicaSet keeps a fixed number of Pods running, forever — that's auto-healing.
- A Deployment creates and manages a ReplicaSet for you automatically.
- Updating a Deployment's image creates a brand-new ReplicaSet and rolls traffic over gradually.
- `kubectl rollout undo` instantly reverts to the previous working version.
- Scaling works the same way on both objects: `kubectl scale ... --replicas=N`.
- In real projects, you create Deployments — not bare ReplicaSets.

Quick Quiz

1. What does a ReplicaSet do if you delete one of its Pods?
1. What object does a Deployment automatically create and manage?
1. Which command starts a rolling update to a new image version?
1. Which command undoes a bad rollout?
1. Name one thing a Deployment can do that a bare ReplicaSet cannot.

■ LAB COMPLETE

You've now built, healed, scaled, updated, and rolled back real Kubernetes workloads — the exact same operations used to run production systems every day.